Vidyavardhini's College of Engineering and Technology, Vasai

Department of Computer Science & Engineering (Data Science)

EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2025-26

Course: Multidisciplinary Minor
Course code: MDC401
Year: SE SEM: IV

Experiment No. 1
AIM: Install XAMPP / WAMP. Install Git and set up a new repository. Learn fundamental Git commands: add, commit, modify, view. Create and manage branches for version control.
Name: Ashish Ramesh Patel
Roll Number: 55
Date of Performance:
Date of Submission:

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	**20**	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	5	3	2
Understanding	5	3	2
Journal work and timely submission.	10	8	4

Checked by

Name of Faculty : Dr. Yogesh Pingle

Aim: Install XAMPP / WAMP. Install Git and set up a new repository. Learn fundamental Git commands: add, commit, modify, view. Create and manage branches for version control.

Theory:

Local Server Environment (XAMPP / WAMP)

Web applications often require a server, database, and scripting language to function properly. Installing a live server for development and testing is neither economical nor practical for beginners. To overcome this limitation, local server environments such as XAMPP and WAMP are used.

XAMPP is a cross-platform software package that provides Apache web server, MySQL (or MariaDB) database, PHP, and Perl in a single installation. It allows developers to host and test dynamic web applications locally.
WAMP performs the same function but is limited to Windows operating systems.

These tools simulate a real hosting environment on a personal computer, enabling students to understand client-server architecture, server-side scripting, and database interaction without an internet connection.

Need for Version Control

During software development, source code is modified frequently. Managing multiple versions manually can lead to data loss, overwriting of files, and confusion. A Version Control System (VCS) solves these problems by maintaining a complete history of changes made to files over time.

Version control allows developers to:

- Track who made changes and when
- Restore previous versions of files
- Work collaboratively on the same project
- Maintain stable and experimental versions simultaneously

Types of Version Control Systems

There are two major types of version control systems:

Centralized Version Control System (CVCS)
In this system, a single central server stores the repository, and all users interact with it. Examples include SVN and CVS. If the central server fails, the entire system becomes unavailable.

Distributed Version Control System (DVCS)
In a distributed system, each user has a complete copy of the repository, including full history. Git is a distributed version control system, making it faster, more reliable, and suitable for collaborative development.

Git as a Distributed Version Control System

Git is an open-source distributed version control system developed by Linus Torvalds. Unlike centralized systems, Git allows developers to work offline and commit changes locally. Synchronization with remote repositories can be done later.

Git efficiently tracks changes using snapshots instead of differences, which improves performance and reliability. Each commit represents a complete state of the project at a given time.

Git Repository Structure

A Git repository consists of:

- Working Directory – where files are edited
- Staging Area – where changes are prepared for commit
- Repository (.git directory) – where commits and metadata are stored

The repository maintains:

- Commit history
- Branch information
- Configuration settings
- Object database

Git Workflow

The standard Git workflow involves three stages:

1. **Modify** – Changes are made in files
2. **Stage** – Changes are added to the staging area
3. Commit – Changes are saved permanently in the repository

This workflow ensures controlled and organized version tracking.

Fundamental Git Commands:

- **git init**
  Initializes a new Git repository in a directory.
- **git add**
  Adds modified or new files to the staging area.
- **git commit**
  Saves staged changes as a snapshot with a descriptive message.
- **git status**
  Displays the current state of the working directory and staging area.
- **git log**
  Shows the history of commits along with commit IDs, author details, and timestamps.

Branching in Git

A branch represents an independent line of development. The default branch is usually named main. Branching allows developers to work on new features, bug fixes, or experiments without affecting the stable version of the project.

Git branches are lightweight and efficient, making branch creation and deletion fast operations.

Branch Management

Branch management includes:

- Creating branches for specific tasks
- Switching between branches
- Merging branches into the main branch
- Deleting obsolete branches

This mechanism enables parallel development and improves productivity in team projects.

Advantages of Git in Software Development

A. Maintains complete project history
B. Enables collaboration among multiple developers
C. Prevents accidental data loss
D. Supports branching and merging
E. Allows rollback to earlier versions
F. Works efficiently with remote repositories such as GitHub

Prerequisites

- Windows/Linux OS
- Internet connection
- Basic command-line knowledge

Procedure

A. Install XAMPP/WAMP

1. Download XAMPP from the official Apache Friends website.
2. Run the installer and select Apache, MySQL, and PHP components.
3. Complete installation and open XAMPP Control Panel.
4. Start Apache and MySQL services. 5. Verify installation by opening http://localhost in a browser.

B. Install and Configure Git

1. Download Git from the official Git website.
2. Install Git with default settings.
3. Open Git Bash and configure user details:
git config --global user.name "Your Name"
git config --global user.email youremail@example.com

C. Create Repository and Perform Git Operations

1. Open Visual Studio Code on the system.
2. Create a new folder named myproject.
3. Open the folder in VS Code
   (File → Open Folder → select myproject).
4. Open the Terminal in VS Code
   (Terminal → New Terminal).
5. Initialize a new Git repository in the project folder:
   git init
6. Create a new HTML file named index.html.
7. Write basic HTML code in index.html and save the file:

```
<!DOCTYPE html>

<html>

<head>

  <title>My First Git Project</title>

</head>

<body>

  <h1>Hello Git</h1>

</body>

</html>
```

8. Check the status of the repository:
   git status
9. Add the HTML file to the staging area:
   git add index.html
10. Commit the staged file with a commit message:
    git commit -m "Add index.html file"
11. Modify the HTML file by adding new content and save it:
    This file is tracked using Git.
12. Check the status to view modified files:
    git status
13. Add the modified file to the staging area:
    git add index.html
14. Commit the updated HTML file:
    git commit -m "Update index.html content"
15. View the commit history:
    git log

Branch Creation and Management:

1. Create a new branch named feature
   git branch feature

2. Switch to the new branch:
   git checkout feature
3. Modify index.html by adding new content and save:
   This content is from feature branch.
4. Add and commit changes in the feature branch:
   git add index.html
   git commit -m "Update index.html in feature branch"
5. Switch back to the main branch:
   git checkout main
6. Merge the feature branch into main:
   git merge feature
7. View all branches:
   git branch

Push Local Repository to Github.

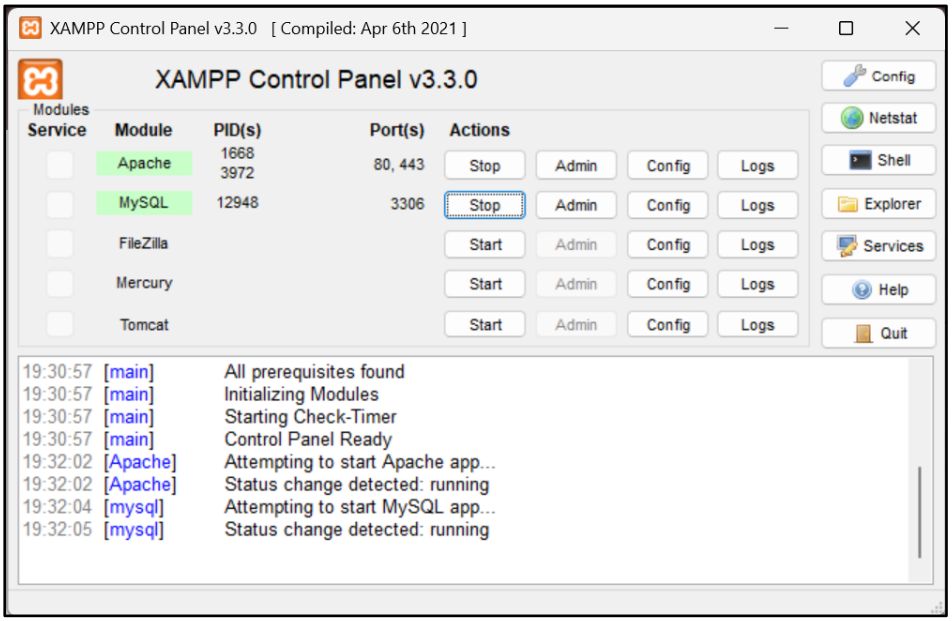
1. Create a new repository on GitHub (without README).
2. Add GitHub repository as remote origin:
   git remote add origin https://github.com/username/repository-name.git
3. Push local main branch to GitHub:
   git branch -M main
   git push -u origin main

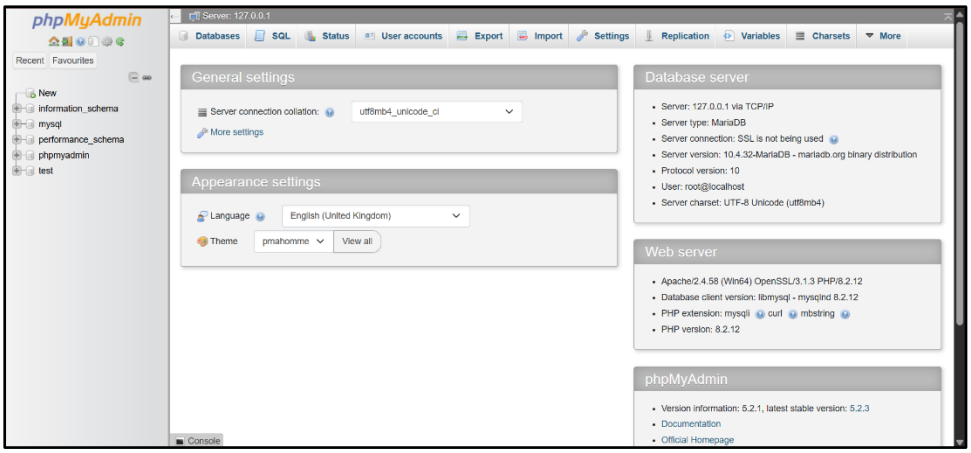
Pull Changes From Github:

1. Make any change directly on GitHub (or from another system).
2. Pull latest changes into local repository:
   git pull origin main
3. View Commit History
   git log

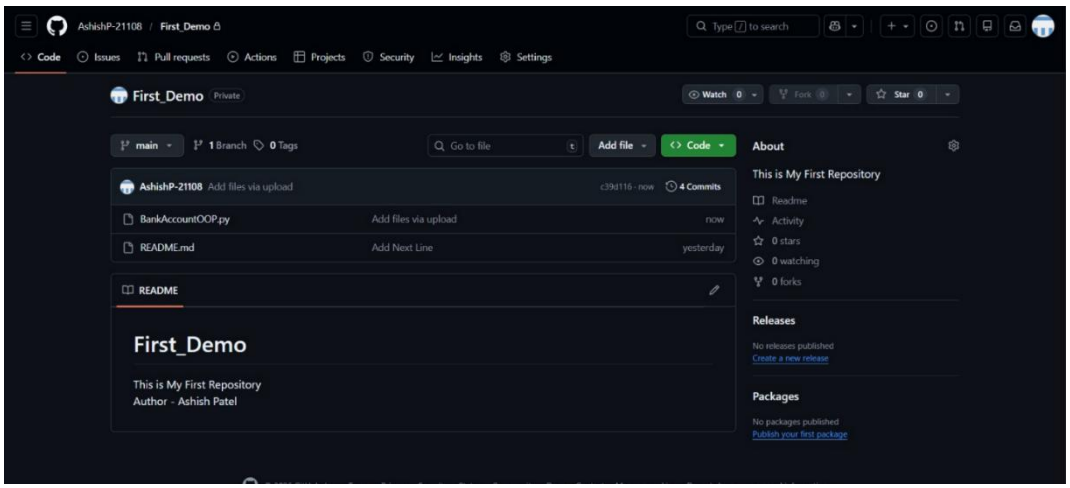
Output:

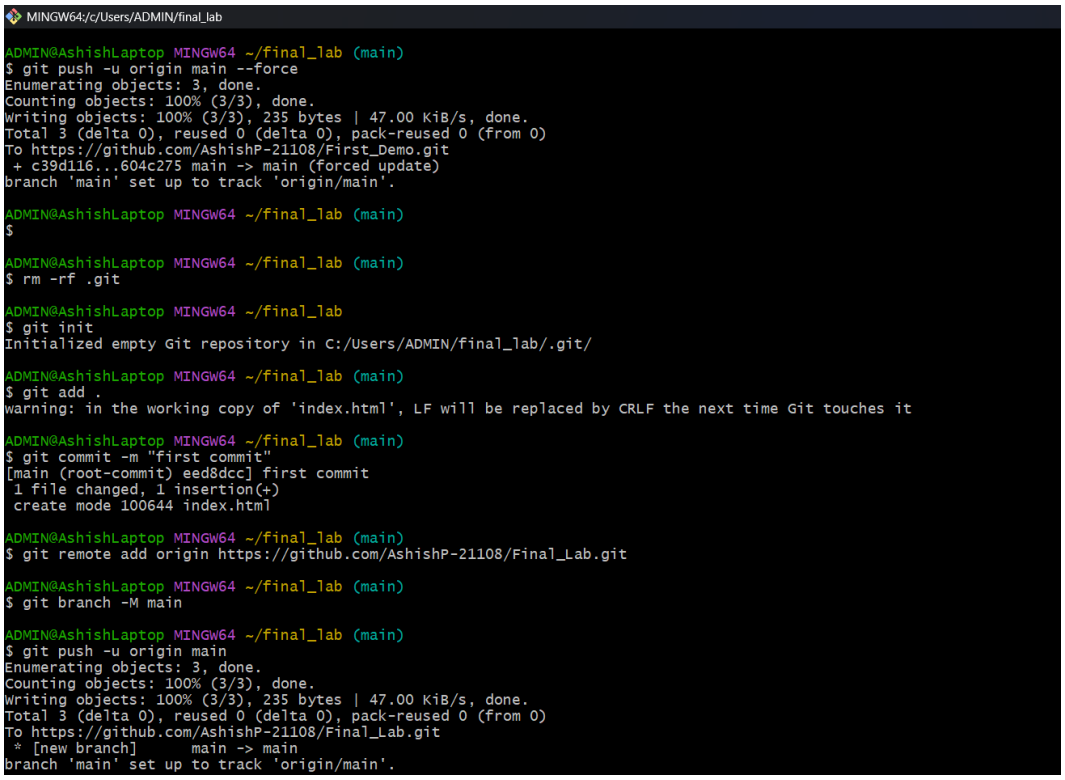
XAMPP installation:





New repository setup





```
MINGW64:/c/Users/ADMIN/final_lab

ADMIN@AshishLaptop MINGW64 ~/final_lab (main)
$ git push -u origin main --force
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 235 bytes | 47.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/AshishP-21108/First_Demo.git
 + c39d116...604c275 main -> main (forced update)
branch 'main' set up to track 'origin/main'.

ADMIN@AshishLaptop MINGW64 ~/final_lab (main)
$

ADMIN@AshishLaptop MINGW64 ~/final_lab (main)
$ rm -rf .git

ADMIN@AshishLaptop MINGW64 ~/final_lab
$ git init
Initialized empty Git repository in C:/Users/ADMIN/final_lab/.git/

ADMIN@AshishLaptop MINGW64 ~/final_lab (main)
$ git add .
warning: in the working copy of 'index.html', LF will be replaced by CRLF the next time Git touches it

ADMIN@AshishLaptop MINGW64 ~/final_lab (main)
$ git commit -m "first commit"
[main (root-commit) eed8dcc] first commit
 1 file changed, 1 insertion(+)
 create mode 100644 index.html

ADMIN@AshishLaptop MINGW64 ~/final_lab (main)
$ git remote add origin https://github.com/AshishP-21108/Final_Lab.git

ADMIN@AshishLaptop MINGW64 ~/final_lab (main)
$ git branch -M main

ADMIN@AshishLaptop MINGW64 ~/final_lab (main)
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 235 bytes | 47.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/AshishP-21108/Final_Lab.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Conclusion

Thus, XAMPP/WAMP and Git were installed successfully. Basic Git commands were executed, and branch management was performed to understand version control operations.

Questions:

1.What is a version control system?

Ans:
Definition: A software tool that records changes to files over time.
Key Points:

- Tracks modifications and history.
- Allows rollback to previous versions.
- Enables collaboration among multiple developers.

2. What is branching in Git?

Ans:
Definition: Branching is creating a separate line of development in Git.
Key Points:

- Used to develop features or fix bugs independently.
- Keeps the main branch stable.
- Branches can be merged back after testing.

3. What is GitHub and how is it used for collaboration?

Ans:
Definition: GitHub is a cloud-based hosting service for Git repositories.
Usage for Collaboration:

- Developers push/pull code to a shared repository.
- Pull Requests allow code review and merging.
- Issues and discussions help manage tasks.
- Enables distributed teamwork across locations.

4. Difference between local repository and remote repository.

Ans:

Aspect	Local Repository	Remote Repository
Location	Stored on developer's personal computer	Hosted on server/cloud (e.g., GitHub)
Usage	For development, commits, and testing	For sharing, syncing, and collaboration
Accessibility	Accessible only to the local user	Accessible to team members via internet
Purpose	Individual work	Team collaboration and backup